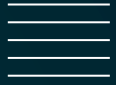




Intro to Cyber Security

By Ahmed El-Qalaawi

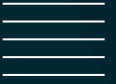
Topics



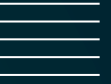
- » Software Development Life Cycle (SDLC)
- » Cyber Crimes
- » Secure Software Development Life Cycle (SSDLC)
- » CIA Triad
- » Authentication, Authorization & Auditing (AAA)
- » Vulnerability, Threat and Risk
- » CVSS Score Metrics
- » OWASP Top 10
- » Cryptography
- » Practice



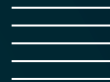
Software Development Life Cycle (SDLC)



Cyber Crimes



Cyber Crimes



Google

Google+ API Data Exposure (2018)

- » One of Google+'s People APIs exposed private profile information of up to 500,000 users to third-party developers.
- » Google fixed the bug immediately, but chose not to disclose it publicly right away, fearing reputational damage and regulatory scrutiny.
- » When the Wall Street Journal revealed the story in October 2018, Google officially acknowledged the breach.

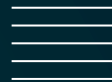
Microsoft

Exchange Server ProxyLogon (2021)

- » Critical attack chain exploiting 4 zero-day vulnerabilities in Microsoft Exchange Server
- » 250,000+ servers compromised globally
- » Pre-authentication remote code execution

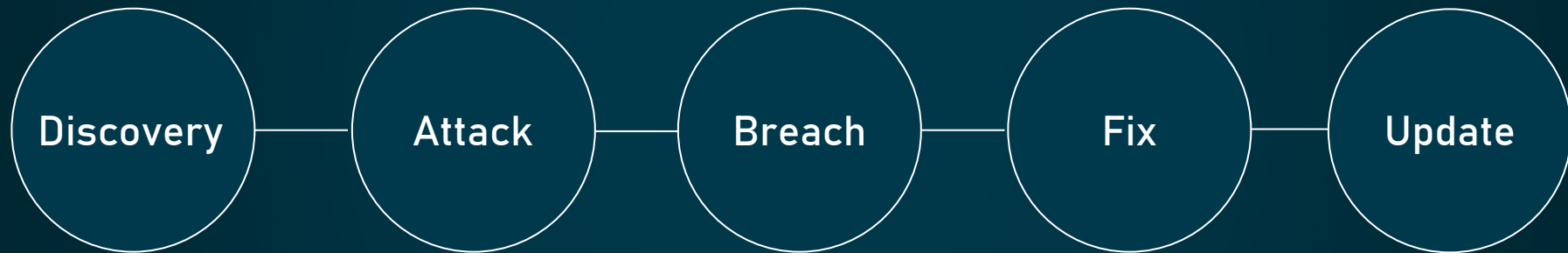


Zero-day Vulnerability Timeline



Vulnerability packaged
into an attack

Vendor works on a fix

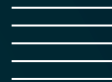


Vulnerability discovered
by hacker

A target is breached

Vendor releases a fix

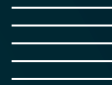
Common Vulnerabilities and Exposures (CVE)



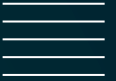
- » CVE is an international system for identifying and naming publicly known cybersecurity vulnerabilities.
- » Each CVE entry provides a unique ID and basic details about a specific security flaw.
- » Format: CVE-YEAR-ID (e.g. CVE-2021-6498)
- » It helps organizations share information and coordinate responses to vulnerabilities more effectively.



Secure Software Development Life Cycle (SSDLC)



CIA Triad



Confidentiality

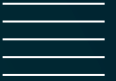


Integrity



Availability

CIA Triad



Confidentiality

- » Cracking Encrypted Data
- » Man In The Middle attack on plain text
- » Data leakage / Unauthorized copying of sensitive data
- » Installing Spyware/Malware on server



Integrity

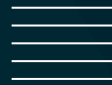
- » Web Penetration for malware insertion
- » Maliciously accessing servers and forging records
- » Unauthorized database scans
- » Remotely controlling zombie systems



Availability

- » Deliberately disrupting a server rooms power supply
- » Flooding a server with too many requests
- » DoS/DDoS attacks
- » Ransomware attacks –
- » Forced encryption of key data

Check knowledge



1. An attacker steals a company's customer database and posts emails publicly.
2. A printer queue is flooded so employees can't print important reports.
3. A developer's config file is changed so prices on the e-commerce site show €0.00.
4. An employee's password is intercepted and used to read private HR files.
5. A ransomware gang encrypts servers and the payroll system stops working.
6. Malware silently modifies financial transaction records to hide theft.
7. A public API endpoint accidentally returns secret API keys to anyone who calls it.
8. An attacker launches a DDoS against the company website, making it unreachable.
9. A trusted sensor's firmware is tampered with so it reports false temperature readings.
10. An insider copies trade-secret documents and sells them to a competitor.



CIA Triad & AAA



Authentication

vs.

Authorization



Who are you?

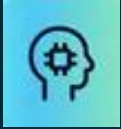
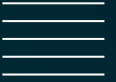
- » Validate the identification of the user.
- » Make the user prove that they are the person they claim to be.



What you allowed to do?

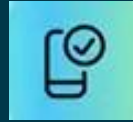
- » Check the user's permissions.
- » What can they access or perform?
- » What can't they access or perform?

Authentication Factors



Something You KNOW

- » Passwords
- » PIN
- » Security Questions



Something You HAVE

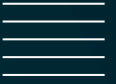
- » SMS OTP
- » Mobile App
- » Security Keys



Something You ARE

- » Fingerprint
- » Face recognition
- » Iris / retina scans

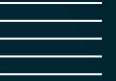
Check knowledge



1. A user enters the wrong password three times and can't log in.
2. An employee successfully logs in but then tries to access the admin dashboard and gets "Access Denied."
3. The system keeps logs of who accessed which files and when.
4. A hacker bypasses the login page and directly visits a secret URL, gaining access without proving who they are.
5. The company reviews logs showing who deleted a record in the database.
6. A mobile app scans the user's fingerprint for verification.
7. The employee uses their ID card badge to open a secure door.
8. The system asks for the user's mother's maiden name as a security question.



Key Terms



Vulnerability

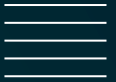


Threat



Risk

Vulnerability vs. Threat vs. Risk



Vulnerability

- » Refers to the weakness of an asset that can be exploited by attackers.
- » It's a bug in hardware or software that is prone to be exploited to cause damage to one of the elements within CIA triad.



Threat

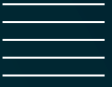
- » Refer to any event that has the potential to bring harm to an organization.
- » Can be natural, intentional or unintentional.
- » Threat assessment techniques are used for understanding threats.



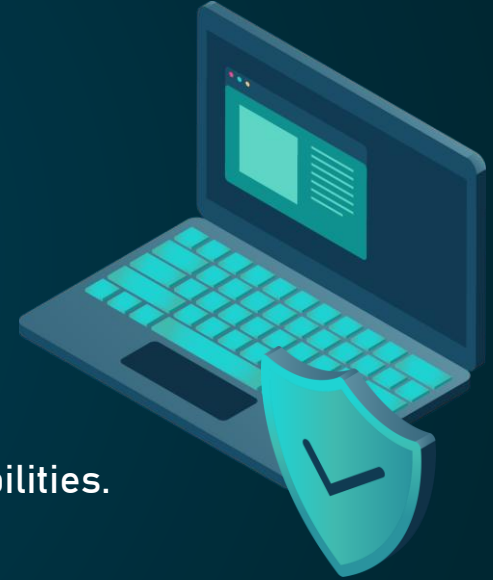
Risk

- » Refers to the potential for loss or damage when a threat exploits a vulnerability.
- » Risk = Threat x Vulnerability.
- » Risk management is key to cyber security.

Vulnerability vs. Threat vs. Risk

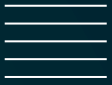


- Asset ➡ What we are trying to protect.
- Threat ➡ What we are trying to protect against.
- Vulnerability ➡ The weakness or gap in our protection effort.
- Risk ➡ The intersection of assets, threats, and vulnerabilities.



$$R = A + T + V$$

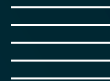
Check knowledge



1. In a cybersecurity context, what might allow an attacker to gain access even without any new tools or exploits?
2. A new ransomware campaign is targeting outdated operating systems — what term best describes the ransomware itself?
3. A company stores passwords in plain text. What cybersecurity concept does this situation mainly illustrate?
4. An organization identifies that a power outage could disrupt its data center. Which of the three terms applies to this situation?
5. When we estimate the possible financial loss from a successful cyberattack, which concept are we dealing with?
6. An attacker discovers a flaw in a web app but has not yet acted on it. Which of the three elements exists right now?
7. A report states: “The likelihood of data loss due to insider misuse is medium.” Which concept is being measured here?
8. If all weaknesses are patched but attackers are still attempting access, which of the three remains present?
9. A phishing campaign is detected targeting multiple organizations. For a company with poor employee training, which part increases most?

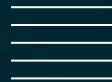


CVSS Score Metrics



Basic Metric Group		Temporal Metric Group	Environmental Metric Group
Exploitability Metrics	Impact Metrics	● Exploit Code Maturity ● Remediation Level ● Report Confidence	● Confidentiality Requirement ● Integrity Requirement ● Availability Requirement ● Modified Base Metrics
● Attack Vector ● Attack Complexity ● Privileges Required ● User Interaction ● Scope	● Compatibility Impact ● Integrity Impact ● Availability Impact ● Scope		

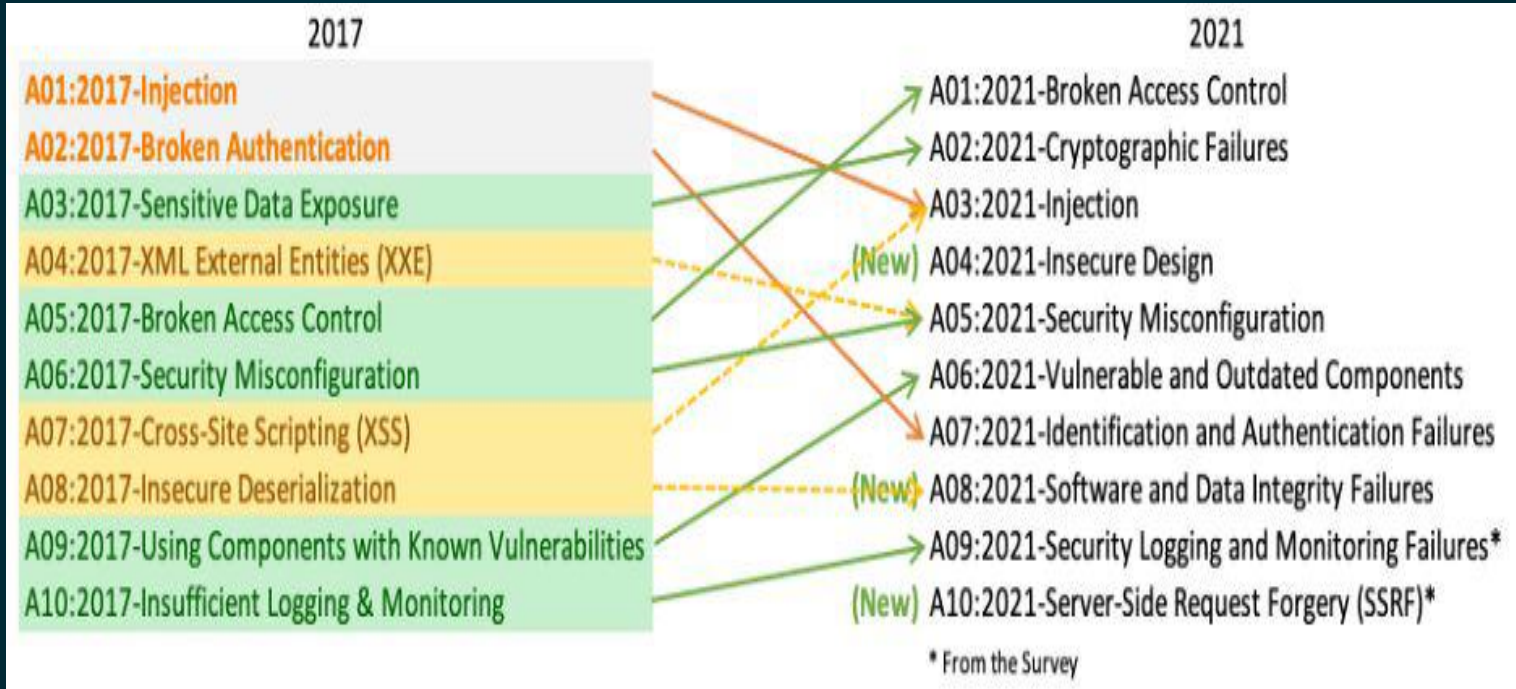
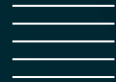
OWASP



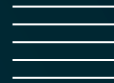
- » OWASP (Open Worldwide Application Security Project) is a non-profit organization focused on improving the security of software.
- » It provides free resources, tools, and guidelines to help developers and security professionals build safer applications.
- » OWASP is best known for its Top Ten list, which highlights the most critical web application security risks.



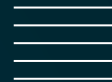
OWASP TOP 10



Let's practice!

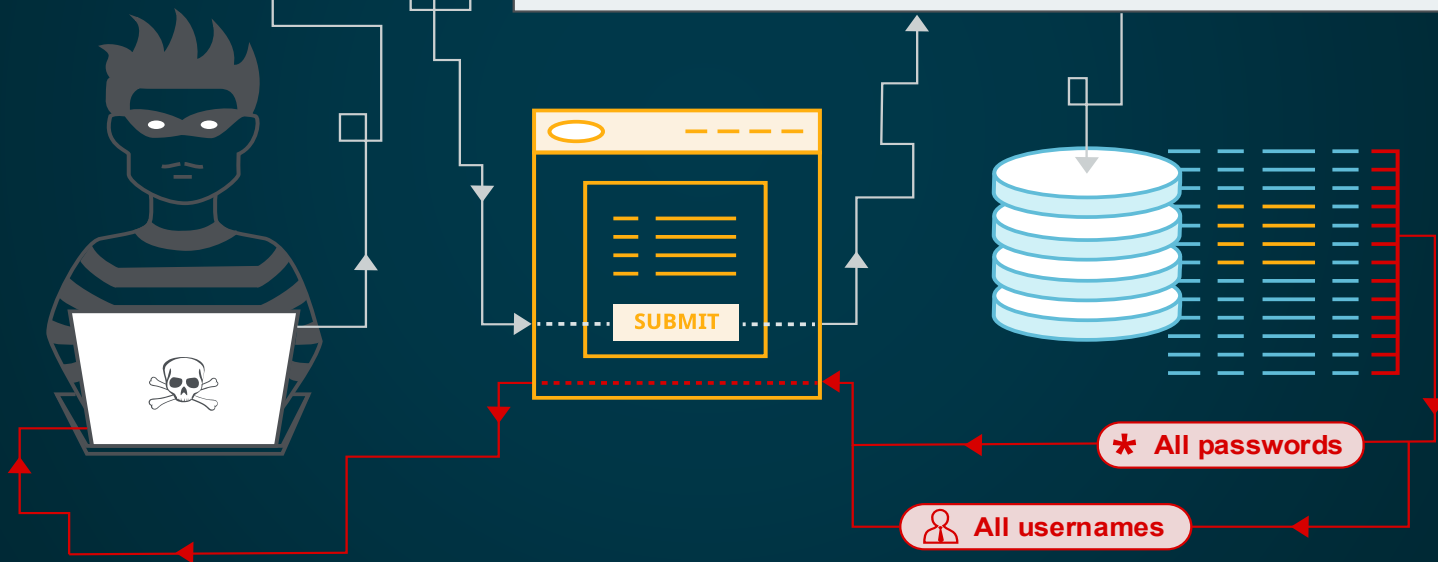


SQL Injection

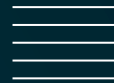


```
' UNION SELECT username, password FROM users--
```

```
SELECT name, description FROM products WHERE category  
= 'Gifts' UNION SELECT username, password FROM users--
```



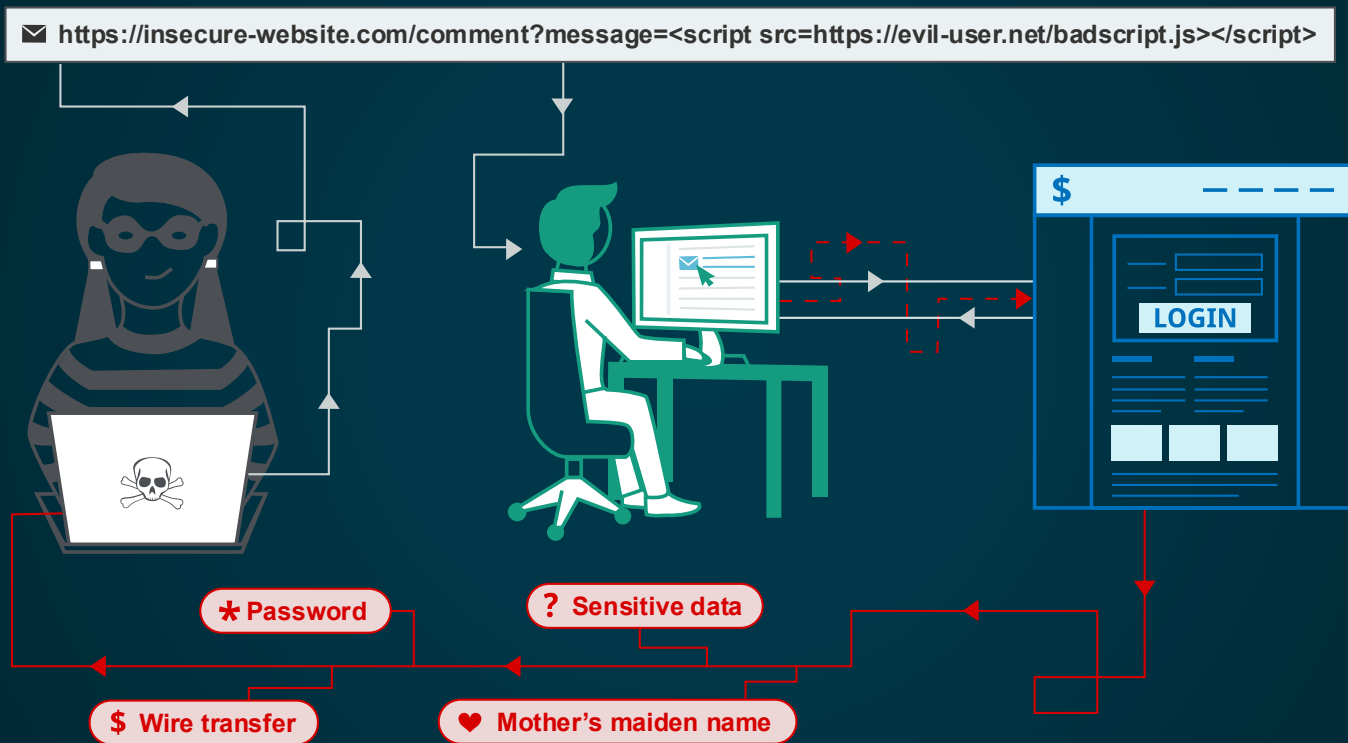
SQL Injection



```
import express from "express";
import mysql from "mysql";
import path from "path";

const getUser = (id: number) => {
  return new Promise((resolve, reject) => {
    const query = "SELECT * FROM users WHERE id="+id;
    db.query(query, (err, rows) => {
      if(err) return reject("")
      if(rows.length === 0) return reject("")
      resolve(rows[0].name)
    })
  });
}
```

XSS

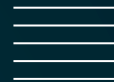


XSS



```
> ITI > Exercises > XSS > JS index.js > ...
1  const express = require('express');
2  const app = express();
3  const PORT = process.env.PORT || 1337;
4
5  app.get('/', (req, res) => {
6    const username = req.query.username || 'stranger';
7    res.setHeader('Content-Type', 'text/html; charset=utf-8');
8    res.send(`
9      <!doctype html>
10     <html>
11       <head><meta charset="utf-8"><title>Hello</title></head>
12       <body>
13         <h1>Hello ${username}</h1>
14       </body>
15     </html>
16   `);
17 });
18
19 app.listen(PORT, '0.0.0.0', () => {
20   console.log(`Vulnerable server listening at http://0.0.0.0:${PORT}`);
21 });
```

Cryptographic Terms



Encoding

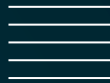
Encryption

Hashing

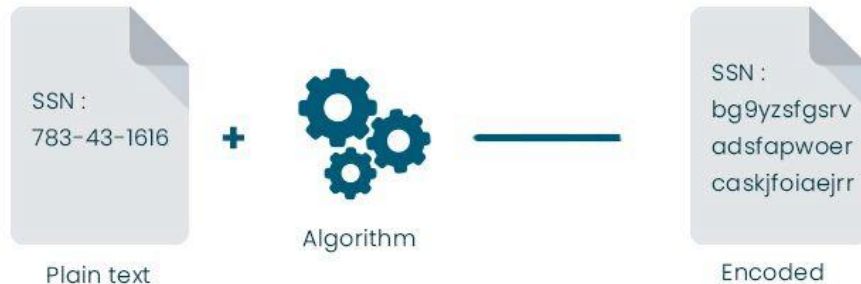
Salting



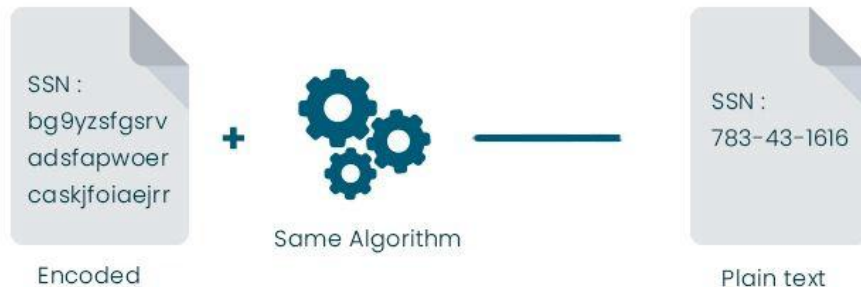
Encoding & Decoding



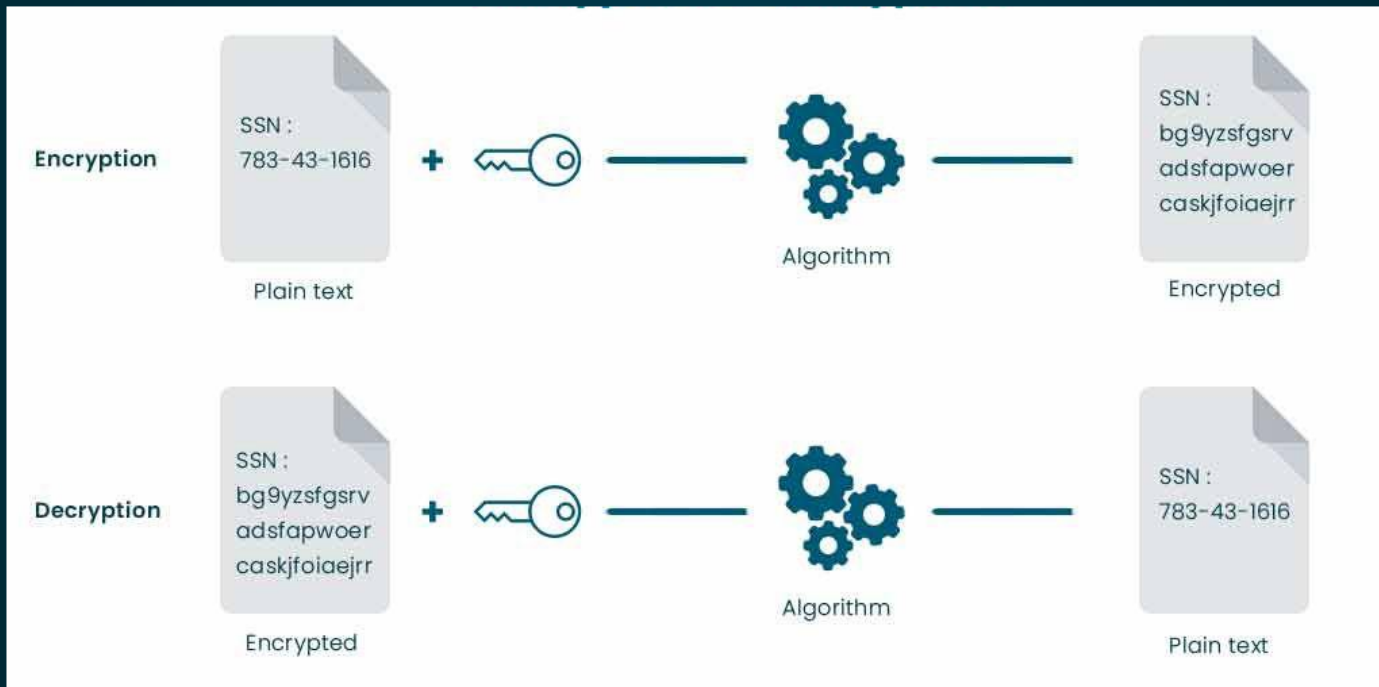
Encoding



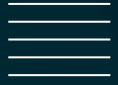
Decoding



Encryption & Decryption



Hashing



Plain Text

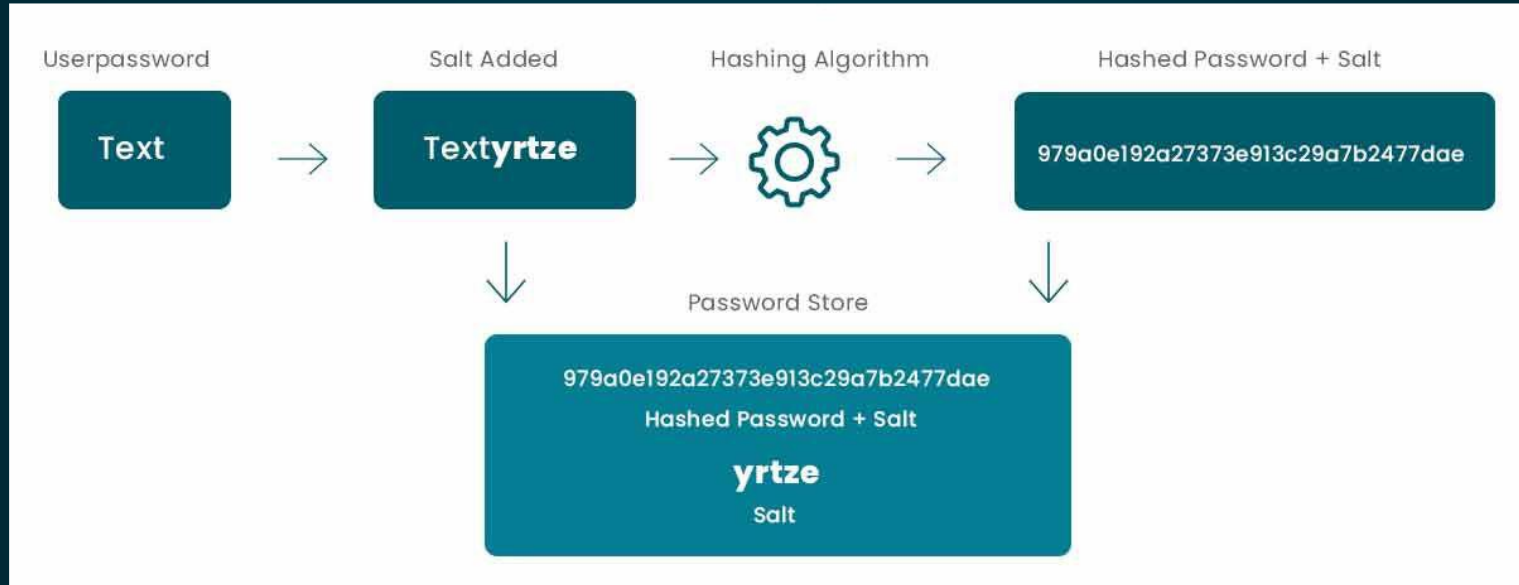
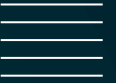


Hash Function

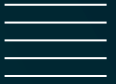


Hashed Text

Salting



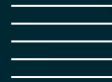
Check knowledge



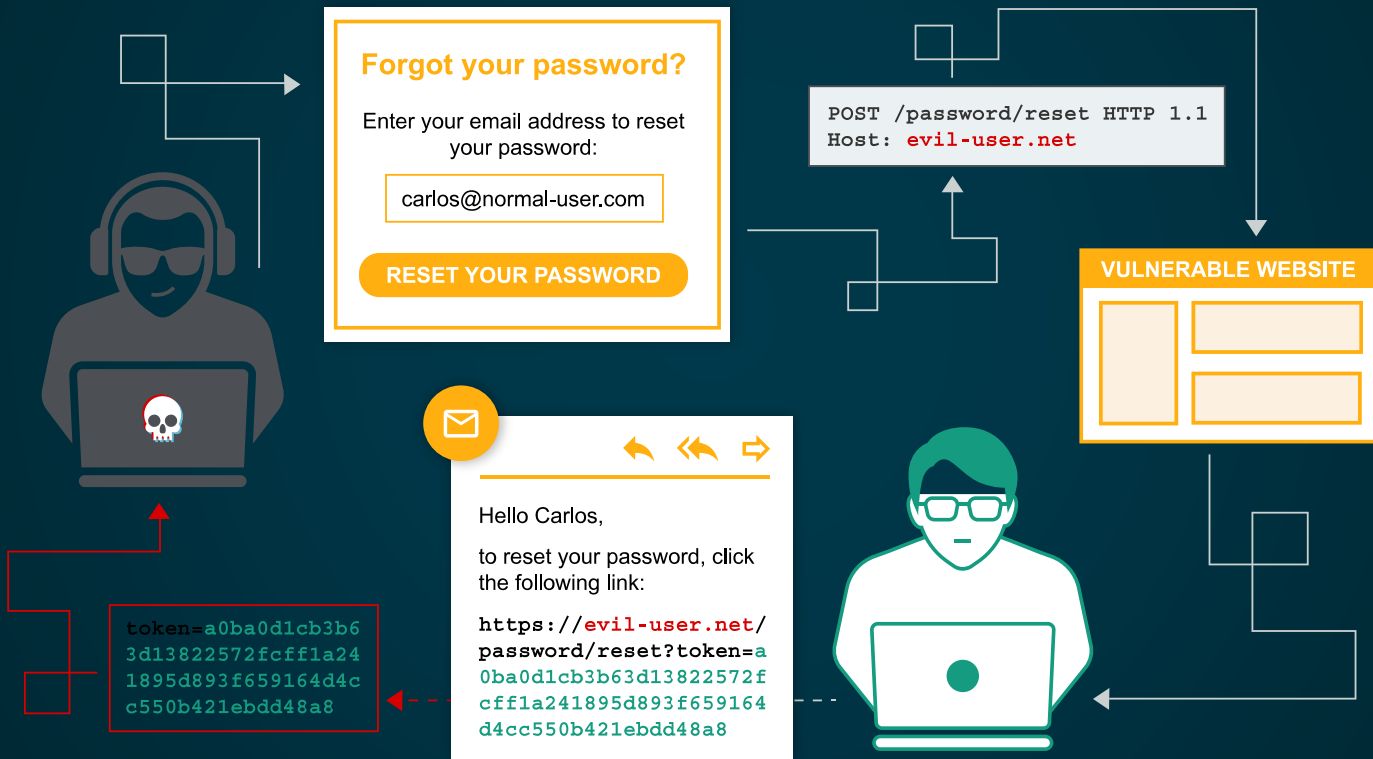
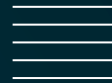
1. A web developer changes data into a different format so it can safely pass through a URL without breaking.
2. A company stores users' passwords in a way that even system administrators can't see them, and the original passwords can't be recovered.
3. A message sent between two servers looks like random characters and can only be understood by the system that holds a special key.
4. Two users who have the same password end up with different stored values in the database.
5. A tool converts binary data into a text form to make it easier to transmit through an email.
6. A hacker steals a file of password hashes but can't use a precomputed rainbow table because random data was added to each password first.
7. The IT team calculates a short fixed-length value from each file to later check if any file was modified.
8. A confidential report is scrambled before being sent to another department, and only the recipient can make it readable again.
9. Someone takes a string that looks like `SGVsbg8gV29ybGQ=` and turns it back into readable text.
10. A web service combines a user's password with unique random data before running it through a one-way algorithm.



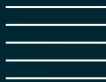
Let's practice!



Broken Authentication



Broken Authentication



```
import express from "express";
import { Request, Response, NextFunction } from 'express';

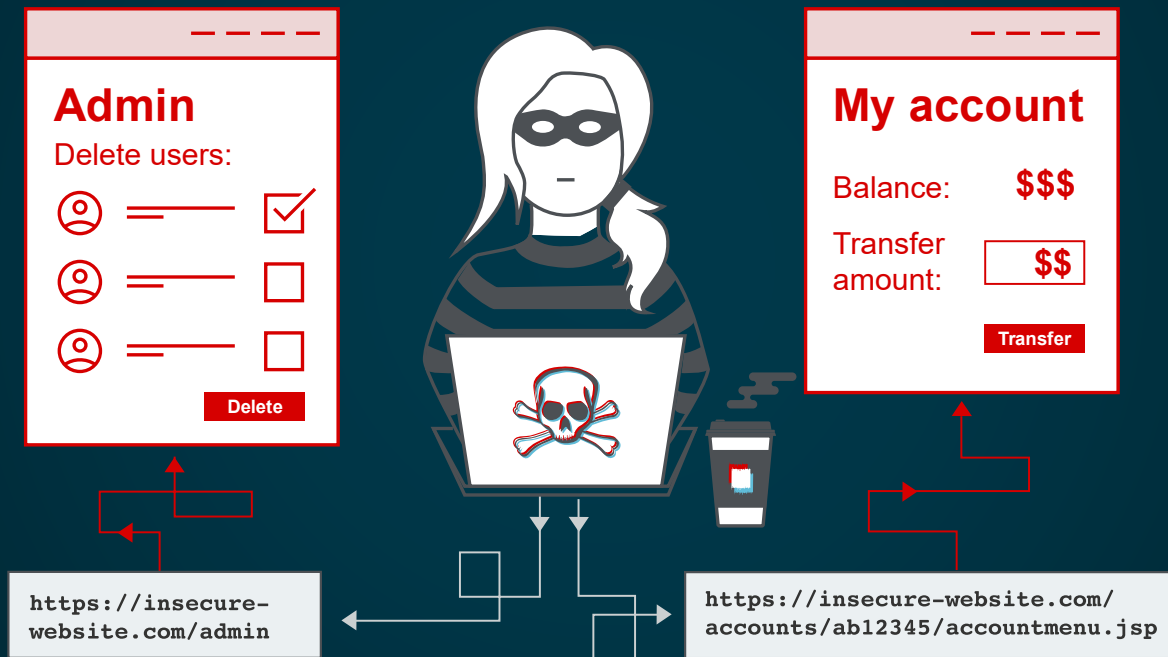
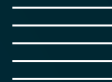
const app = express();

function basicAuth(req: Request, res: Response,
  next: NextFunction) {
  if (!req.headers.authorization || req.headers.authorization.indexOf('Basic ') === -1) {
    return res.status(401).json({ message: 'Missing Authorization Header' });
  }
  const base64Credentials = req.headers.authorization.split(' ')[1];
  const credentials = Buffer.from(base64Credentials, 'base64').toString('ascii');
  const [username, password] = credentials.split(':');
  if (!(username === process.env.myuser || password === process.env.mypassword)) {
    return res.status(401).json({ message: 'Invalid Authentication Credentials' });
  }

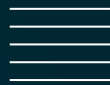
  next();
}

app.use(basicAuth);
```

Broken Access Control



Broken Access Control



: > ITI > Exercises > Broken Access > JS index.js > ...

```
1  const http = require('http');
2
3  http.createServer((req, res) => {
4    const raw = req.headers.cookie || '';
5    const cookies = {};
6    raw.split(';').forEach(p => {
7      const i = p.indexOf('=');
8      if (i > -1) cookies[p.slice(0,i).trim()] = decodeURIComponent(p.slice(i+1).trim());
9    });
10
11    const isAdmin = (cookies.Admin || '').toLowerCase() === 'true';
12
13    res.writeHead(200, {'Content-Type': 'text/html; charset=utf-8'});
14    res.end(
15      '<html><body>' +
16      '<h1>Welcome</h1>' +
17      (isAdmin ? '<h2>Secret: 42-ALPHA</h2>' : '<p>You are not authorized to view this content.</p>') +
18      '</body></html>'
19    );
20  }).listen(1337, '0.0.0.0', () => console.log('vuln http://0.0.0.0:1337'));
```



THANK



YOU